Tugas 9: Tugas Praktikum Mandiri 09

Revani – 0110224111 [1]

[1] Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224111@student.nurulfikri.ac.id

Abstract. Pada tugas praktikum mandiri 9 ini, dilakukan penerapan metode klasifikasi Naive Bayes untuk memprediksi jenis diagnosis kanker payudara berdasarkan berbagai parameter pengukuran jaringan seperti radius, tekstur, perimeter, dan area. Tujuan dari latihan ini adalah memahami bagaimana Naive Bayes bekerja dalam mengolah data numerik, bagaimana proses preprocessing mempengaruhi performa model, dan bagaimana hasil prediksi dapat dianalisis menggunakan confusion matrix serta classification report. Dataset yang digunakan adalah data kanker payudara, yang berisi label diagnosis "M" untuk malignant (ganas) dan "B" untuk benign (jinak). Setelah melalui tahap pembersihan data, encoding label, pembagian data, standardisasi, pelatihan model, hingga evaluasi, model Naive Bayes berhasil memberikan performa klasifikasi yang cukup baik. Praktikum ini juga memberikan gambaran nyata bagaimana machine learning dapat membantu proses deteksi dini penyakit dengan memanfaatkan pola dari data medis.

1. Penjelasan Hasil Praktikum Mandiri

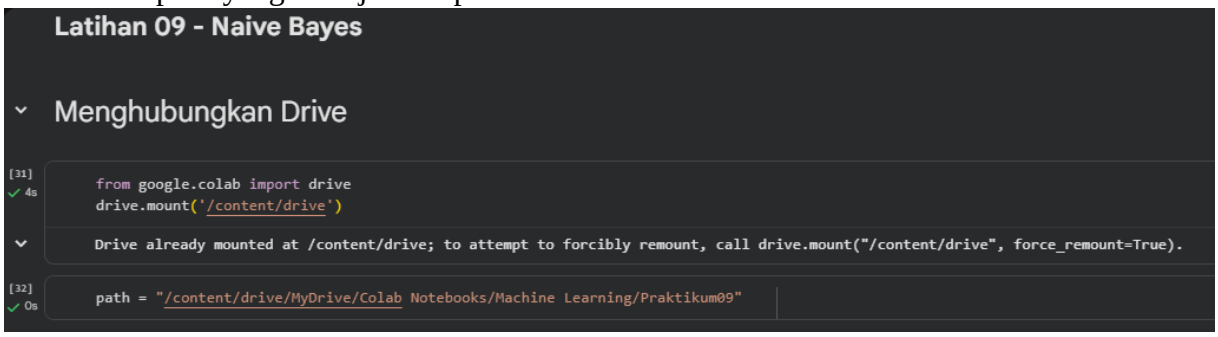
Pada praktikum ini, Hasil dari proses klasifikasi menunjukkan bahwa algoritma Naive Bayes mampu melakukan prediksi diagnosis kanker payudara dengan tingkat akurasi tinggi. Setelah data dibersihkan dari kolom yang tidak relevan dan dilakukan encoding pada kolom diagnosis, model dilatih pada data training yang telah distandarisasi. Hasil evaluasi menggunakan confusion matrix dan classification report menunjukkan bahwa model dapat membedakan antara kanker jinak dan ganas dengan cukup baik. Secara keseluruhan, implementasi ini membuktikan bahwa Naive Bayes menjadi salah satu algoritma yang cocok untuk dataset dengan banyak fitur numerik seperti dataset kanker. Berikut ini adalah hasil praktikum yang saya kerjakan pada praktikum mandiri 09.

2. Penjelasan Kode Program

```
from google.colab import drive
drive.mount('/content/drive')
path = "/content/drive/MyDrive/Colab Notebooks/Machine Learning/Praktikum09"
```

Langkah awal yang dilakukan adalah menghubungkan Google Colab dengan Google Drive agar notebook bisa membaca file CSV yang disimpan dalam folder Drive. Variabel path digunakan sebagai alamat folder tempat dataset kanker disimpan. Output yang dihasilkan yaitu

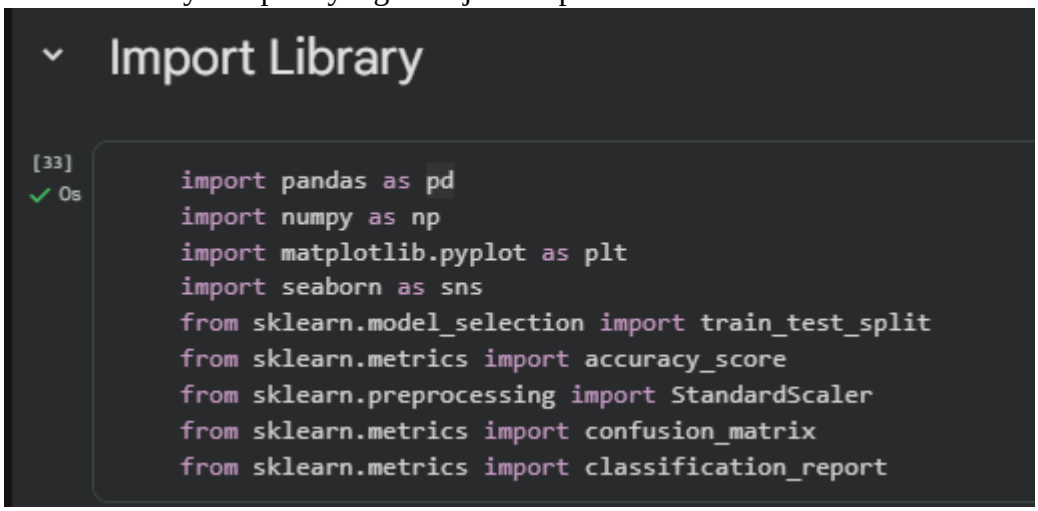
colab akan meminta izin akses Drive. Jika berhasil, akan muncul pesan bahwa Drive berhasil di-mount seperti yang ditunjukkan pada Gambar 1.



Gambar 1. Pada bagian ini ditunjukkan menghubungkan *gdrive*

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Bagian ini memuat semua library yang diperlukan, seperti pandas untuk membaca data, numpy untuk operasi numerik, seaborn/matplotlib untuk visualisasi, dan library scikit-learn untuk pemodelan Naive Bayes seperti yang ditunjukkan pada Gambar 2.



Gambar 2. Pada bagian ini ditunjukkan untuk mengimpor library.

```
cancer_data = pd.read_csv(path + '/Data/cancer.csv')
```

Bagian ini untuk membaca dataset kanker.

```
cancer_data.head()
cancer_data.tail()
```

Bagian ini untuk menampilkan 5 baris pertama dan 5 baris terakhir.

```
cancer_data.info()
```

Bagian ini untuk menunjukkan informasi terkait tipe data, jumlah kolom, dan apakah ada missing value.

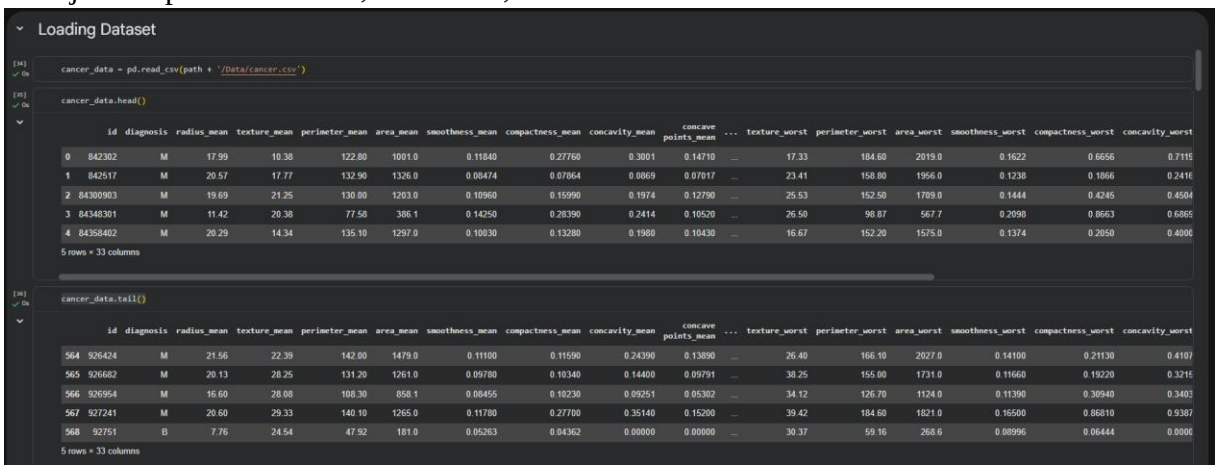
```
cancer_data.describe()
```

Bagian ini memberikan statistik deskriptif seperti mean,min,max, dll.

```
cancer_data.shape
```

Bagian ini menunjukkan jumlah baris dan kolom.

Maka output yang dihasilkan yaitu dataset ini memiliki 569 baris dan 33 kolom. Kolom Unnamed: 32 terlihat kosong karena seluruh isi NaN. Diagnosis berupa huruf M (Malignant / ganas) dan B (Benign / Jinak). Serta semua kolom fitur lain berupa numerik seperti yang ditunjukkan pada Gambar 3, Gambar 4, dan Gambar 5.



Loading Dataset

```
cancer_data = pd.read_csv(path + '/Data/cancer.csv')
```

```
cancer_data.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33	184.60	2019.0	0.1622	0.6656	0.7115
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41	158.80	1956.0	0.1238	0.1866	0.2416
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53	152.50	1709.0	0.1444	0.4245	0.4504
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50	98.87	567.7	0.2098	0.8663	0.6869
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67	152.20	1575.0	0.1374	0.2050	0.4000

5 rows × 33 columns

```
cancer_data.tail()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	...	26.40	166.10	2027.0	0.14100	0.21130	0.4107
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	...	38.25	155.00	1731.0	0.11660	0.19220	0.3215
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	...	34.12	126.70	1124.0	0.11390	0.30940	0.3403
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	...	39.42	184.60	1821.0	0.16500	0.86810	0.9387
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	...	30.37	59.16	268.6	0.08996	0.06444	0.0000

5 rows × 33 columns

Gambar 3. Pada bagian ini ditunjukkan untuk membaca dan menampilkan dataset.

```
[37]
✓ 0s
cancer_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

Gambar 4. Pada bagian ini ditunjukkan untuk informasi dataset.

```
[38]
✓ 0s
cancer_data.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	...	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	c
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	...	25.677223	107.261213	880.583128	0.132369	0.254265	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	...	6.146258	33.602542	569.356993	0.022832	0.157336	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	...	12.020000	50.410000	185.200000	0.071170	0.027290	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	...	21.080000	84.110000	515.300000	0.116600	0.147200	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	...	25.410000	97.660000	686.500000	0.131300	0.211900	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	...	29.720000	125.400000	1084.000000	0.146000	0.339100	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	...	49.540000	251.200000	4254.000000	0.222600	1.058000	

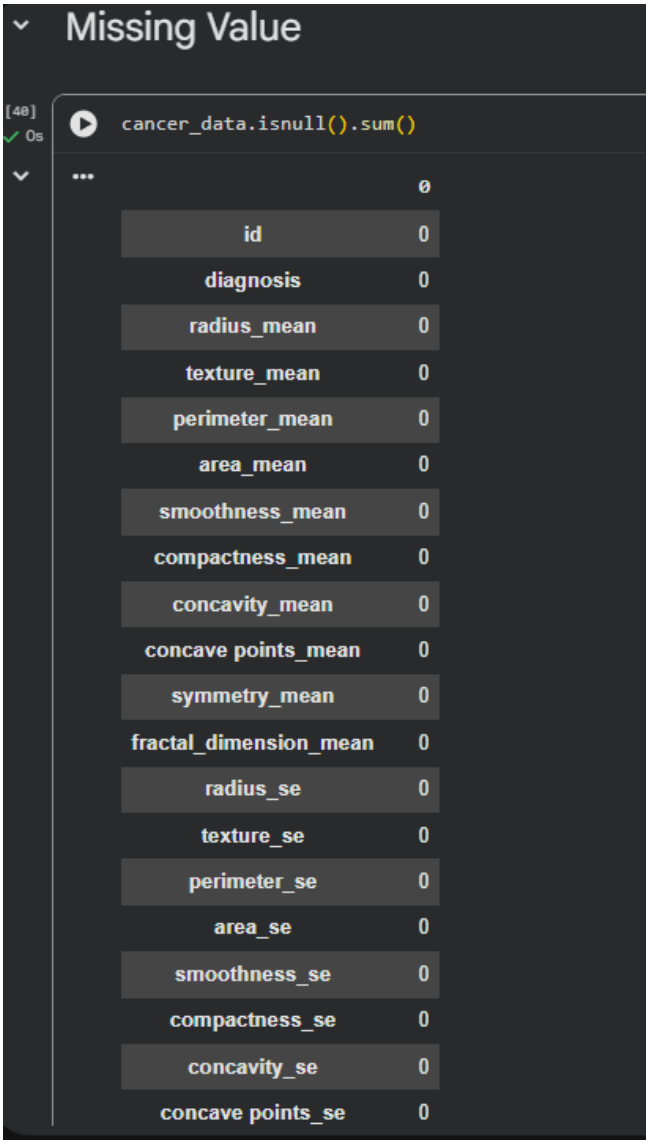
8 rows × 32 columns

```
[39]
✓ 0s
cancer_data.shape

(569, 33)
```

Gambar 5. Pada bagian ini ditunjukkan untuk statistic deskriptif dan menunjukan baris kolom.

```
cancer_data.isnull().sum()
```

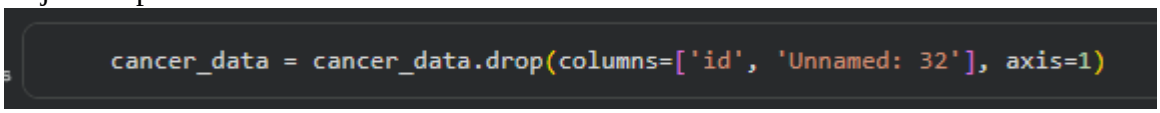
Fungsi ini menghitung berapa banyak nilai kosong pada masing-masing kolom. Output yang dihasilkan yaitu semua kolom memiliki nol missing value, kecuali Unnamed: 32 yang memiliki 569 missing value (kolom kosong) seperti yang ditunjukkan pada Gambar 6.



Gambar 6. Pada bagian ini ditunjukkan untuk missing value.

```
cancer_data = cancer_data.drop(columns=['id', 'Unnamed: 32'], axis=1)
```

Kolom id dan Unnamed: 32 tidak berhubungan dengan diagnosis, sehingga dihapus agar tidak mengganggu model. Outputnya yaitu dataset akan berkurang menjadi 31 kolom seperti yang ditunjukkan pada Gambar 7.



Gambar 7. Pada bagian ini ditunjukkan untuk menghapus kolom tidak relevan.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
cancer_data['diagnosis_encoded'] = le.fit_transform(cancer_data['diagnosis'])
```

Diagnosis berupa huruf B/M harus diubah menjadi angka apabila B berarti 0 dan M berarti 1, jadi kolom baru Bernama diagnosis_encoded ditambahkan ke dataset. Output yang dihasilkan yaitu kolom baru berisi angka 0 dan 1 seperti yang ditunjukkan pada Gambar 8.

```
# encode kolom diagnosis
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
cancer_data['diagnosis_encoded'] = le.fit_transform(cancer_data['diagnosis'])
```

Gambar 8. Pada bagian ini ditunjukkan encoding diagnosis.

```
X = cancer_data.drop(columns=['diagnosis', 'diagnosis_encoded'])
y = cancer_data['diagnosis_encoded']

print("\nData setelah Prepocessing\n")
print(cancer_data.head())
```

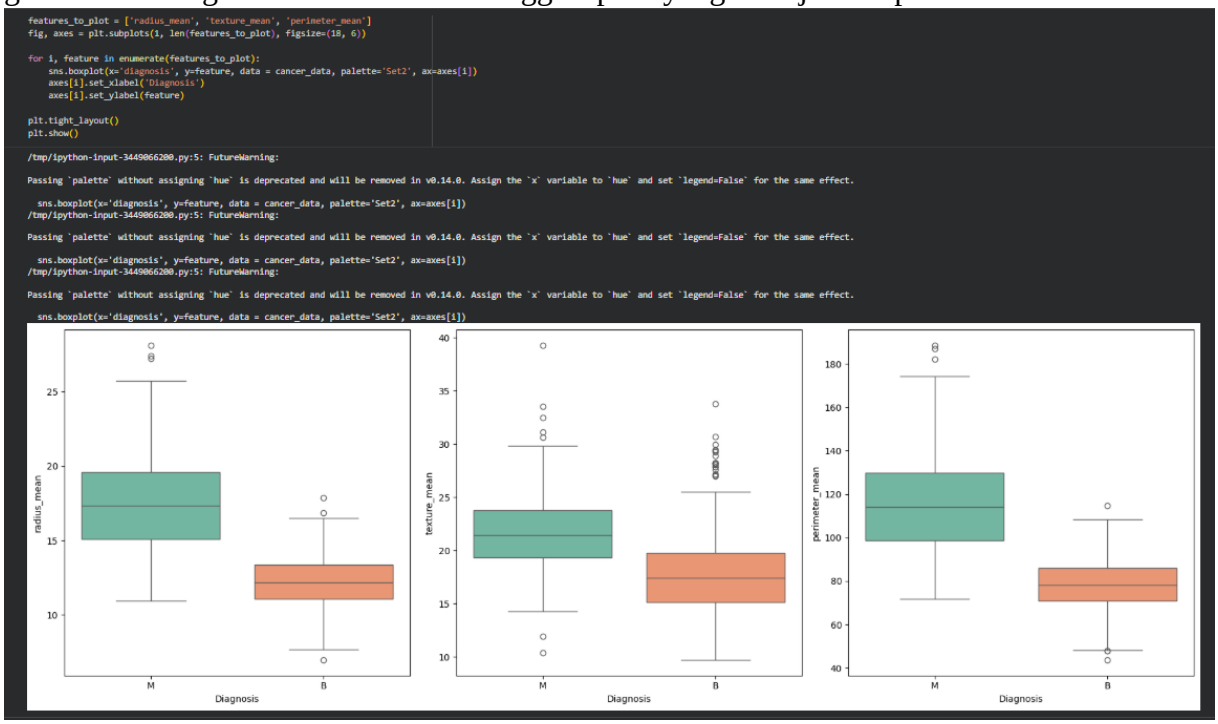
X berisi seluruh fitur kecuali diagnosis dan y hanya berisis diagnosis encoded sebagai target klasifikasi. Output yang dihasilkan yaitu dataset fitur dan target dipisahkan dengan benar seperti yang ditunjukkan pada Gambar 9.

```
# menentukan fitur dan target
X = cancer_data.drop(columns=['diagnosis', 'diagnosis_encoded'])
y = cancer_data['diagnosis_encoded']

print("\nData setelah Prepocessing\n")
print(cancer_data.head())


Data setelah Prepocessing

  diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0         M        17.99         10.38          122.80     1001.0
1         M        20.57         17.77          132.90     1326.0
2         M        19.69         21.25          130.00     1203.0
3         M        11.42         20.38           77.58      386.1
4         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   symmetry_mean  ...  texture_worst  perimeter_worst  area_worst  \
0         0.2419  ...          17.33           184.60      2019.0
1         0.1812  ...          23.41           158.80      1956.0
2         0.2069  ...          25.53           152.50      1709.0
3         0.2597  ...          26.50            98.87       567.7
4         0.1809  ...          16.67           152.20      1575.0

   smoothness_worst  compactness_worst  concavity_worst  concave points_worst  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   symmetry_worst  fractal_dimension_worst  diagnosis_encoded
0          0.4601                  0.11890                  1
1          0.2750                  0.08902                  1
2          0.3613                  0.08758                  1
3          0.6638                  0.17300                  1
4          0.2364                  0.07678                  1

[5 rows x 32 columns]
```

Gambar 9. Pada bagian ini menentukan fitur dan target.

```
features_to_plot = ['radius_mean', 'texture_mean', 'perimeter_mean']
fig, axes = plt.subplots(1, len(features_to_plot), figsize=(18, 6))

for i, feature in enumerate(features_to_plot):
    sns.boxplot(x='diagnosis', y=feature, data = cancer_data, palette='Set2', ax=axes[i])
    axes[i].set_xlabel('Diagnosis')
    axes[i].set_ylabel(feature)

plt.tight_layout()
plt.show()
```

Membuat boxplot untuk melihat apakah pasien kanker jinak dan ganas memiliki distribusi fitur yang berbeda. Outputnya yaitu akan tampil tiga boxplot yang menunjukkan bahwa fitur kanker ganas cenderung memiliki nilai lebih tinggi seperti yang ditunjukkan pada Gambar 10.



Gambar 10. Pada bagian ini ditunjukkan visualisasi outlier / boxplot.

```
#Pemisahan Data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#scaling fitur
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(f"\nUkuran Data Training: {X_train.shape}")
print(f"Ukuran Data Testing: {X_test.shape}")
```

Dataset dipisahkan menjadi 80% data training dan 20% data testing. Output yang dihasilkan yaitu data training 455 dan data testing 114. Kode StandardScaler menstandarkan data agar semua fitur memiliki skala yang sama (mean=0, std=1). Hal ini penting untuk Naive Bayes. Jadi ouput X_train dan X_test berubah menjadi array yang sudah terstandarisasi seperti yang ditunjukkan pada Gambar 11.

```
#Pemisahan Data (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#scaling fitur
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(f"\nUkuran Data Training: {X_train.shape}")
print(f"Ukuran Data Testing: {X_test.shape}")

Ukuran Data Training: (455, 30)
Ukuran Data Testing: (114, 30)
```

Gambar 11. Pada bagian ini membagi data train test dan scaling data.

```
#inisialisasi model
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)

#prediksi pada data uji
test_pred_nb = model.predict(X_test)

print("\nAkurasi Naive Bayes berhasil dilatih dan membuat prediksi.")
```

Membuat model GaussianNB karena semua fitur numerik dan melatihnya menggunakan data training. Serta Model memprediksi diagnosis pasien pada data testing dengan output yang dihasilkan yaitu Variabel test_pred_nb berisi daftar prediksi: 0 dan 1 seperti yang ditunjukkan pada Gambar 12.

Pelatihan Model Naive Bayes

```python
#inisialisasi model
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)

#prediksi pada data uji
test_pred_nb = model.predict(X_test)

print("\nAkurasi Naive Bayes berhasil dilatih dan membuat prediksi.")
```

```
Akurasi Naive Bayes berhasil dilatih dan membuat prediksi.
```

Gambar 12. Pada bagian ini ditunjukkan pelatihan model dan prediksi data uji.

```python
#perhitungan akurasi
accuracy_nb = accuracy_score(y_test, test_pred_nb)
cm_nb = confusion_matrix(y_test, test_pred_nb)
report_nb = classification_report(y_test, test_pred_nb)
print(f"\nAkurasi Naive Bayes: {accuracy_nb:4f}")

print("\nConfusion Matrix Naive Bayes:")
print(cm_nb)

print("\nClassification Report (0 = Jinak, 1 = Ganas)")
print(report_nb)
```

Kode ini untuk menghitung akurasi model, membuat confusion matrix dan menampilkan precision, recall, f1-score. Output yang ditampilkan yaitu angka akurasi, matriks perbandingan prediksi benar atau salah, serta laporan klasifikasi. Biasanya akurasu mencapai 90%+ seperti yang ditunjukkan pada Gambar 13.

Evaluasi Model

[47]
✓ 0s

```
#perhitungan akurasi
accuracy_nb = accuracy_score(y_test, test_pred_nb)
cm_nb = confusion_matrix(y_test, test_pred_nb)
report_nb = classification_report(y_test, test_pred_nb)
print(f"\nAkurasi Naive Bayes: {accuracy_nb:4f}")

print("\nConfusion Matrix Naive Bayes:")
print(cm_nb)

print("\nClassification Report (0 = Jinak, 1 = Ganas)")
print(report_nb)
```

```
Akurasi Naive Bayes: 0.964912

Confusion Matrix Naive Bayes:
[[70  1]
 [ 3 40]]

Classification Report (0 = Jinak, 1 = Ganas)
              precision    recall  f1-score   support

           0       0.96      0.99      0.97        71
           1       0.98      0.93      0.95        43

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114
```

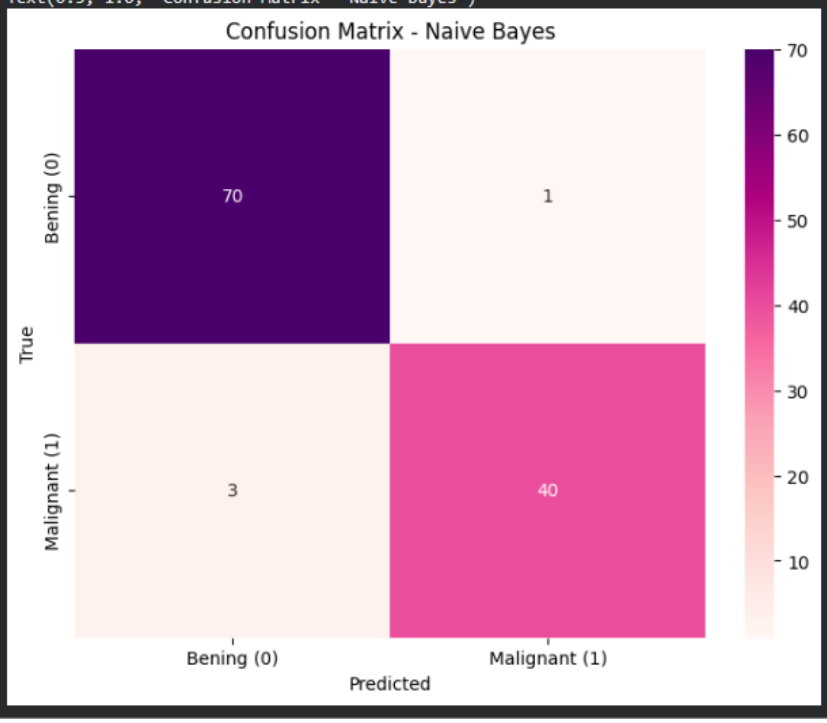
Gambar 13. Pada bagian ini ditunjukkan evaluasi model.

```
class_name = ['Bening (0)', 'Malignant (1)']

plt.figure(figsize=(8, 6))
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='RdPu', xticklabels=class_name, yticklabels=class_name)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix - Naive Bayes')
```

Kode ini untuk menampilkan confusion matrix dalam bentuk heatmap berwarna agar mudah dipahami. Outputnya Heatmap menampilkan jumlah prediksi benar dan salah antara kelas 0 dan 1 seperti yang ditunjukkan pada Gambar 14.

```
class_name = ['Bening (0)', 'Malignant (1)']

plt.figure(figsize=(8, 6))
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='RdPu', xticklabels=class_name, yticklabels=class_name)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix - Naive Bayes')
```

```
Text(0.5, 1.0, 'Confusion Matrix - Naive Bayes')
```



Gambar 14. Pada bagian ini ditunjukkan visualisasi confusion matrix.

3. Kesimpulan dan Hasil Implementasi

Dari hasil implementasi Model Naive Bayes dari praktikum ini sangat berguna untuk mendeteksi kanker payudara secara cepat berdasarkan berbagai ciri jaringan (radius, tekstur, perimeter, dan sebagainya). Implementasi ini dapat diterapkan dalam dunia nyata sebagai sistem pendukung keputusan dalam bidang kesehatan, terutama radiologi atau analisis hasil biopsi. Walaupun sederhana, model ini dapat memberikan prediksi awal yang membantu dokter membuat keputusan lanjutan.

Dari praktikum ini dapat disimpulkan bahwa algoritma Naive Bayes, khususnya GaussianNB, mampu bekerja dengan sangat baik pada dataset kanker payudara yang berisi fitur numerik. Setelah dilakukan preprocessing seperti penghapusan kolom tidak relevan, encoding diagnosis, dan standardisasi data, model dapat mencapai akurasi tinggi dan memberikan prediksi yang cukup akurat untuk membedakan kanker jinak dan ganas. Dengan proses yang mengikuti alur dalam praktikum 09, tugas mandiri ini menunjukkan bahwa Naive Bayes merupakan algoritma yang cepat, sederhana, namun sangat efektif untuk tugas klasifikasi medis.

4. Link GitHub (Praktikum dan Tugas Mandiri 09)
https://github.com/revani18/ML_2025_Revani_3AI01/tree/1a1214b6b5a523feb7b6458a7f8559ac6f05e053/Praktikum09

Praktikum:
https://github.com/revani18/ML_2025_Revani_3AI01/blob/1a1214b6b5a523feb7b6458a7f8559ac6f05e053/Praktikum09/Notebooks/praktikum9.ipynb
Latihan:
https://github.com/revani18/ML_2025_Revani_3AI01/blob/1a1214b6b5a523feb7b6458a7f8559ac6f05e053/Praktikum09/Notebooks/latihan09.ipynb